

Telepulse

A Production-Inspired Big Data Engineering Platform for Urban Mobility Analytics

1. Introduction

Project Overview

TelePulse is an end-to-end Big Data Engineering platform designed to process, analyze, and visualize large-scale telecommunications activity across the city of Milan.

The project demonstrates how modern data engineering technologies can be integrated into a complete analytics pipeline—from raw CSV files to business-ready dashboards.

Originally developed as part of the **NTI Big Data Training Program**, TelePulse follows production-inspired engineering practices including:

- Containerized infrastructure using Docker
 - Distributed storage using HDFS
 - Large-scale data processing using Apache Spark
 - Data warehousing using Apache Hive
 - Business intelligence using Power BI
-

2. Project Motivation

Modern cities continuously generate enormous volumes of telecommunications data.

Every phone call, text message, and internet connection contributes valuable information about:

- Urban activity
- Population movement
- Business districts
- Residential neighborhoods
- Communication behavior
- Digital infrastructure usage

However, this information is usually stored as raw transactional records that provide very little value on their own.

The purpose of TelePulse is to transform these raw records into analytical datasets capable of answering real business questions through a fully automated data engineering pipeline.

3. Dataset and Business Questions

3.1. Dataset Overview

TelePulse uses the **Telecom Italia Big Data Challenge** dataset, which contains anonymized, aggregated mobile network activity across **Milan, Italy**, during **November 2013**. The data is organized into daily CSV files representing telecommunications activity over geographic grid cells while preserving user privacy. These files are merged and processed through the TelePulse pipeline to generate analytical insights.

3.2. Dataset Files

The project uses two primary datasets together with two geographical reference files.

```
data/
├── raw/
│   ├── sms-call-internet-mi-2013-11-01.csv
│   ├── sms-call-internet-mi-2013-11-02.csv
│   ├── ...
│   └── sms-call-internet-mi-2013-11-07.csv
│
│   ├── mi-to-provinces-2013-11-01.csv
│   ├── mi-to-provinces-2013-11-02.csv
│   ├── ...
│   └── mi-to-provinces-2013-11-07.csv
│
│   └── ISTAT_census_variables_2011.csv
├── merged/
│   ├── sms_call_internet_all.csv
│   ├── mi_to_provinces_all.csv
│   ├── milano-grid.geojson
│   └── Italian_provinces.geojson
└── merge_datasets.py
```

3.2.1. Activity Dataset

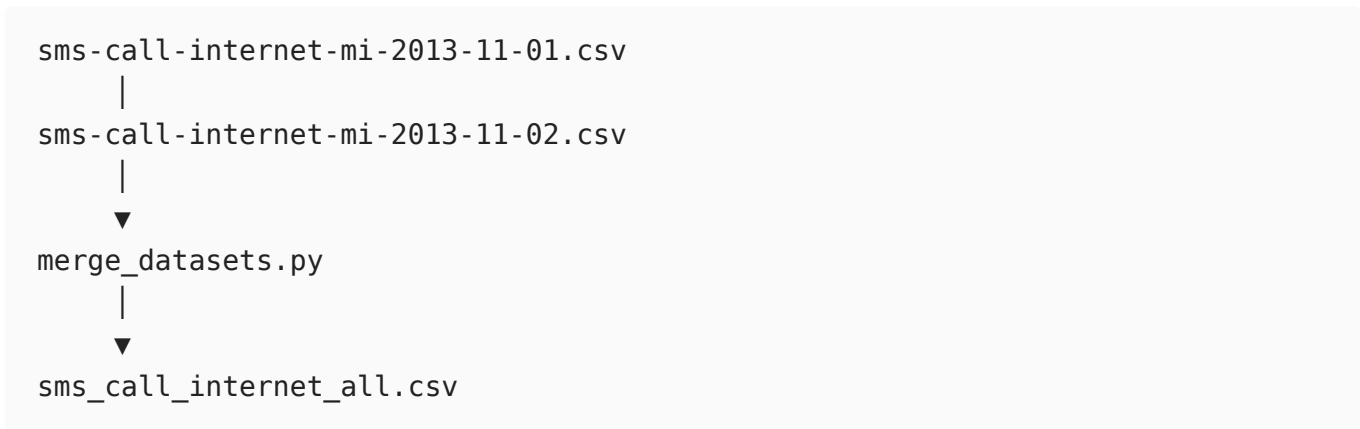
Overview

- The activity dataset contains aggregated telecommunications activity for every Milan grid cell during each time interval.
- Each row represents the activity occurring inside one cell during a specific timestamp.

Schema

Field	Description
datetime	Timestamp of the recorded activity
CellID	Unique identifier of the Milan grid cell
countrycode	Country identifier
smsin	Number of incoming SMS messages
smsout	Number of outgoing SMS messages
callin	Number of incoming phone calls
callout	Number of outgoing phone calls
internet	Internet traffic generated within the cell

Merge



3.2.2. Province Interaction Dataset

Overview

- The second dataset measures communication strength between Milan grid cells and surrounding Italian provinces.
- Instead of measuring local activity, it measures regional interaction.

Schema

Field	Description
datetime	Timestamp
CellID	Milan grid cell
provinceName	Italian province
cell2Province	Communication from Milan cell to province
Province2cell	Communication from province to Milan cell

Merge

```
mi-to-provinces-2013-11-01.csv
|
mi-to-provinces-2013-11-02.csv
|
▼
merge_datasets.py
|
▼
mi_to_provinces_all.csv
```

This preprocessing step greatly simplifies downstream processing by allowing Spark to read one consolidated dataset instead of multiple daily files.

3.3 Business Questions

TelePulse transforms raw telecommunications activity into meaningful business insights capable of answering questions such as:

- Which areas of Milan are the most active?
- Which locations generate the highest internet traffic?
- Which cells produce the largest number of phone calls?
- Which neighborhoods appear to be residential?
- Which neighborhoods appear to be business districts?
- Which areas communicate with the largest variety of surrounding provinces?
- Which locations should be considered urban hotspots?

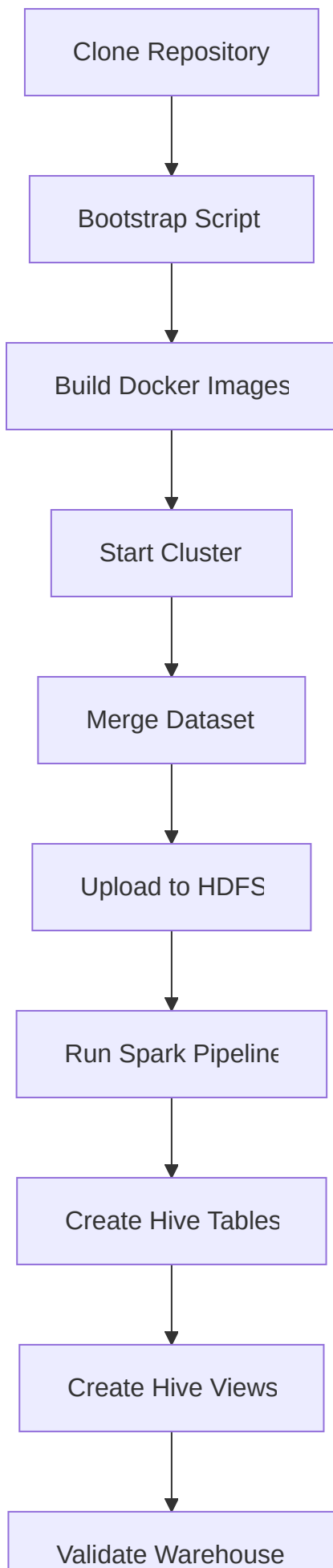
Instead of manually exploring millions of individual records, analysts can query concise analytical datasets that summarize these patterns.

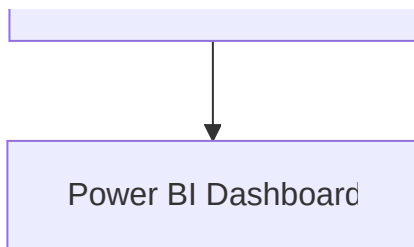
4. System Architecture

4.1. Technology Stack

Component	Responsibility
Docker	Provides a reproducible execution environment
HDFS	Distributed storage
Spark	Distributed data processing
Hive	Data warehouse
Power BI	Dashboard visualization

4.2. Architecture Diagram





The bootstrap script automates nearly the entire workflow, allowing developers to build the complete environment using a single command.

5. Repository Organization

The repository is divided into several logical components.

```
TelePulse/  
  
├─ data/  
├─ infrastructure/  
├─ processing/  
├─ scripts/  
├─ docs/  
├─ requirements.txt  
└─ README.md
```

Each directory contains a specific part of the overall platform.

- **data/** stores preprocessing utilities, raw datasets, merged datasets, and HDFS upload scripts.
 - **infrastructure/** contains the Docker images and Docker Compose configuration used to build the Hadoop ecosystem.
 - **processing/** contains the Spark analytics pipeline and Hive warehouse scripts.
 - **scripts/** contains automation scripts such as image building and project bootstrapping.
 - **docs/** contains the technical documentation for the project.
-

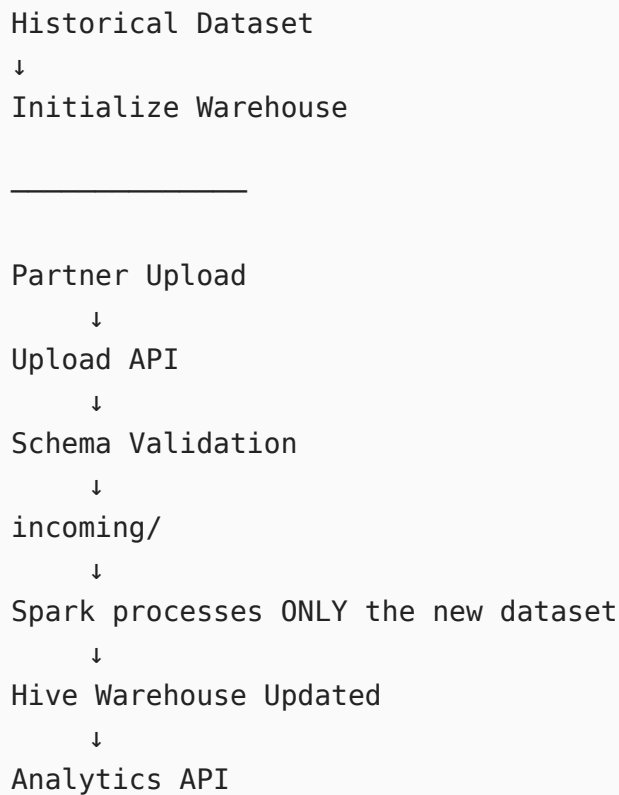
6. Future Work

6.1 Version 2

Version 2 transforms TelePulse from a historical analytics project into an incremental data platform.

- Instead of rebuilding the warehouse from scratch, the system processes only newly uploaded datasets while preserving historical data.
- The warehouse is initialized once using the historical dataset.
- After initialization, every new partner upload updates the warehouse incrementally.

Incremental Pipeline



Improvements

- Incremental processing
- File validation before ingestion
- Event-driven execution
- Historical data preservation
- Faster processing
- Reduced computational cost
- Production-ready ingestion workflow

6.2. Version 3

Version 3 focuses on workflow automation and production orchestration.

- The analytical pipeline remains batch-oriented, but every stage becomes automatically managed.

- Major additions include:
 - Apache Airflow
 - dbt
 - Monitoring
 - Automated validation
 - Scheduled execution

Airflow Pipeline

```
graph TD; A[Partner Upload] --> B[Validate Dataset]; B --> C[Spark Processing]; C --> D[Update Hive]; D --> E[Run dbt Models]; E --> F[Validation]; F --> G[Analytics API];
```

Partner Upload
↓
Validate Dataset
↓
Spark Processing
↓
Update Hive
↓
Run dbt Models
↓
Validation
↓
Analytics API

Improvements

- Automatic scheduling
- Dependency management
- Retry mechanisms
- Monitoring
- Alerting
- Easier maintenance